

PHP

Professional Complete Guide

[Syntax](#) · [OOP](#) · [Namespaces](#) · [Composer](#) · [PDO](#) · [Regex](#) · [Sessions](#) · [Security](#) · [REST API](#)

Book 1 of 15 · W3Schools Professional Series · 2026

Table of Contents

#	Chapter	Key Topics
1	Introduction & Syntax	Tags, echo, comments, variables, data types, casting
2	Strings & Numbers	All string/math functions, regex, formatting
3	Control Flow	if/switch/match, all loop types, break/continue
4	Functions	Default, variadic, closures, arrow functions, recursion
5	Arrays	Indexed, associative, multi-dim, all array functions
6	OOP Basics	Classes, objects, properties, methods, constructors
7	OOP Advanced	Inheritance, abstract, interfaces, traits, static
8	Namespaces & Composer	Autoloading, PSR-4, using packages
9	Error Handling	try/catch/finally, custom exceptions, error levels
10	File System	fopen, fread, fwrite, directories, file info
11	Superglobals & Forms	GET/POST/FILES, validation, sanitization
12	Sessions & Cookies	session_start, \$_SESSION, setcookie
13	PDO & MySQL	Connect, prepare, CRUD, transactions, fetch modes
14	Security & Regex	SQL injection, XSS, CSRF, bcrypt, preg functions
15	Exercises	20 mixed exercises with full answers

Chapter 1: Introduction & Syntax

PHP tags, variables, data types, type casting

What is PHP?

PHP (Hypertext Preprocessor) is a widely-used, open-source, server-side scripting language designed for web development. PHP code is executed on the server and the result is returned to the browser as plain HTML. PHP files use the **.php** extension and can contain HTML, CSS, JavaScript, and PHP code together.

- PHP runs on the server — users never see PHP source code
- Works with virtually all major databases (MySQL, PostgreSQL, SQLite, etc.)
- Free and open source — runs on Linux, Windows, macOS
- Powers WordPress, Drupal, Laravel, Symfony, and much more
- Current stable: PHP 8.3 (2024)

PHP Tags & Output

php_syntax.php

```
<?php
// Standard PHP tag – always use this
echo "Hello, World!";
echo "Multiple", " values", " in one echo";

// print – returns 1, slightly slower than echo
print "Hello!";

// Short echo tag (only if short_open_tag=On)
// <?= "Hello"; ?>

// Heredoc – multi-line string with variable parsing
$name = "Alice";
$text = <<<EOT
Hello, $name!
This is a heredoc string.
EOT;

// Nowdoc – like heredoc but NO variable parsing
$text2 = <<<'EOT'
Hello, $name! // $name is NOT replaced here
EOT;

var_dump($text); // detailed type info
print_r($text); // human-readable
```

Variables & Naming Rules

Variables in PHP start with \$. They are case-sensitive. A variable name must start with a letter or underscore, followed by letters, numbers, or underscores.

```
<?php
$name = "Alice"; // string
$age = 30; // integer
$price = 19.99; // float
$is_active = true; // boolean
$nothing = null; // NULL

// Multiple assignment
$a = $b = $c = 0;

// Variable variables
$varName = "city";
$$varName = "Berlin"; // creates $city = "Berlin"
echo $city; // Berlin

// Constants
define("MAX_SIZE", 100);
const APP_NAME = "MyApp";
echo MAX_SIZE; // 100
echo PHP_VERSION; // built-in constant
echo PHP_INT_MAX; // largest integer
echo PHP_EOL; // newline for current OS
```

Data Types

Type	Description	Example	Check function
String	Text	<code>\$s = "hello";</code>	<code>is_string(\$s)</code>
Integer	Whole number	<code>\$i = 42;</code>	<code>is_int(\$i)</code>
Float	Decimal	<code>\$f = 3.14;</code>	<code>is_float(\$f)</code>
Boolean	true/false	<code>\$b = true;</code>	<code>is_bool(\$b)</code>
Array	Collection	<code>\$a = [1,2,3];</code>	<code>is_array(\$a)</code>
Object	Class instance	<code>\$o = new Cls;</code>	<code>is_object(\$o)</code>
NULL	No value	<code>\$n = null;</code>	<code>is_null(\$n)</code>
Resource	External ref	<code>\$r = fopen(...);</code>	<code>is_resource(\$r)</code>

Type Casting & Juggling

```
<?php
// Explicit casting
$str = "42 apples";
$num = (int)$str; // 42
$flt = (float)"3.14px"; // 3.14
$boo = (bool)""; // false (empty string = false)
$boo2= (bool)"0"; // false ("0" = false)
$boo3= (bool)"false"; // TRUE! (non-empty non-"0" = true)
$arr = (array)"hello"; // ["hello"]

// settype()
$x = "5";
settype($x, "integer"); // $x is now int 5

// Checking types
var_dump(is_numeric("42")); // true
var_dump(is_numeric("42.5")); // true
var_dump(is_numeric("42abc")); // false

// intval(), floatval(), strval()
intval("0x1A", 16); // 26 (hex)
intval("0b1010", 2); // 10 (binary)
intval("017", 8); // 15 (octal)
```

Chapter 2: Strings & Numbers

String functions, regex, math, formatting

String Functions — Complete Reference

```
<?php
$s = " Hello, World! ";
strtolower($s); strtoupper($s); ucfirst($s); ucwords($s);
trim($s); ltrim($s); rtrim($s);
strlen($s); strpos($s,"World"); strrpos($s,"l");
substr($s,2,5); // "Hello" substr($s,-8,6); // "World!"
str_replace("World","PHP",$s); str_ireplace("world","PHP",$s);
explode(" ", $s); implode("|", $arr); str_split("Hello",2);
str_pad("5",3,"0",STR_PAD_LEFT); str_repeat("ab",3);
sprintf("%.2f",9.5); number_format(1234567.89,2,".",",");
strcmp("a","b"); strcasecmp("A","a");
htmlspecialchars("<b>"); strip_tags("<b>Hi</b>");
base64_encode("Hi!"); base64_decode("SGkh");
md5("pw"); sha1("pw"); // Do NOT use for passwords!
```

Regular Expressions (PCRE)

PHP uses Perl-Compatible Regular Expressions via the **preg_*** family of functions.

```

<?php
// preg_match – test if pattern matches
$email = "alice@example.com";
if (preg_match('/^[w.+-]+@[w-]+\.[w.]+$/', $email)) {
    echo "Valid email";
}

// preg_match with capture groups
$date = "2026-05-10";
preg_match('/^(\d{4})-(\d{2})-(\d{2})$/', $date, $matches);
// $matches = ["2026-05-10", "2026", "05", "10"]
echo $matches[1]; // 2026 (year)

// Named capture groups
preg_match('/^(?P<year>\d{4})-(?P<month>\d{2})$/', $date, $m);
echo $m["year"]; // 2026

// preg_match_all – find ALL matches
$html = "<a href='a.html'>A</a> <a href='b.html'>B</a>";
preg_match_all('/href=\'([^\']+)\'/', $html, $all);
// $all[1] = ["a.html", "b.html"]

// preg_replace
$clean = preg_replace('/\s+/', " ", "too many spaces");
// "too many spaces"

// preg_split
$parts = preg_split('/[\s,]+/', "one, two three");
// ["one", "two", "three"]

// Common patterns
// /\d+$/ – digits only
// /^[a-zA-Z]+$/ – letters only
// /^[w.-]+@[w-]+\.[a-z]{2,}$/ – email
// /^(+\d{1,3})?\d\s-{7,15}$/ – phone
// /^(?=.*[A-Z])(?=.*\d){8,}$/ – strong password

```

Math Functions

```
<?php
// Basic
abs(-7); // 7
ceil(4.1); // 5
floor(4.9); // 4
round(4.567, 2); // 4.57
round(4.5); // 5 (rounds half up)
fmod(10, 3); // 1.0 (float modulus)
intdiv(10, 3); // 3 (integer division PHP7+)

// Power & root
sqrt(16); // 4
pow(2, 10); // 1024
log(M_E); // 1
log(100, 10); // 2
log10(1000); // 3
log2(8); // 3

// Min / Max
max(3, 7, 1, 9); // 9
min(3, 7, 1, 9); // 1
max([5,2,8,1]); // 8 (array)
clamp($v, 0, 100); // PHP 8.4 (or use min/max)

// Random
rand(1, 100); // random int 1-100
mt_rand(1, 100); // Mersenne Twister (faster)
random_int(1, 100); // cryptographically secure
shuffle($array); // shuffle array in place

// Constants
M_PI; // 3.14159265...
M_E; // 2.71828182...
M_SQRT2; // 1.41421356...
INF; // infinity
NAN; // not a number
```

Chapter 3: Control Flow

if/switch/match, loops, break, continue, goto

Conditionals

```
<?php
// if / elseif / else
$age = 20;
if ($age >= 18) {
    echo "Adult";
} elseif ($age >= 13) {
    echo "Teen";
} else {
    echo "Child";
}

// Ternary operator
$status = ($age >= 18) ? "Adult" : "Minor";

// Short ternary (Elvis operator)
$name = $_GET["name"] ?: "Guest"; // use left if truthy

// Null coalescing (PHP 7+)
$user = $_GET["user"] ?? "Guest"; // use left if not null
$user = $a ?? $b ?? $c ?? "default"; // chain

// Null coalescing assignment (PHP 7.4+)
$config["debug"] ??= false; // set only if null/undefined

// Spaceship operator (PHP 7+) – for sorting
echo (1 <=> 2); // -1 (left < right)
echo (2 <=> 2); // 0 (equal)
echo (3 <=> 2); // 1 (left > right)
```

switch vs match


```
<?php
// switch – loose comparison (==)
$day = "Mon";
switch ($day) {
case "Mon":
case "Tue":
echo "Early week"; break;
case "Fri":
echo "TGIF!"; break;
default:
echo "Midweek";
}

// match – strict comparison (===), returns value (PHP 8)
$code = 404;
$message = match($code) {
200, 201 => "Success",
301, 302 => "Redirect",
404 => "Not Found",
500 => "Server Error",
default => "Unknown",
};
echo $message; // Not Found

// match with no-match exception
try {
$val = match("x") { "a" => 1, "b" => 2 };
} catch (\UnhandledMatchError $e) {
echo "No match found";
}
```

Loops

```

<?php
// for
for ($i = 0; $i < 5; $i++) { echo $i; }

// while
$i = 0;
while ($i < 5) { echo $i++; }

// do...while (runs at least once)
$i = 10;
do { echo $i; $i++; } while ($i < 10); // prints 10

// foreach
$fruits = ["apple", "banana", "cherry"];
foreach ($fruits as $fruit) { echo $fruit; }

// foreach with key
$person = ["name"=>"Alice", "age"=>30];
foreach ($person as $key => $value) {
    echo "$key: $value\n";
}

// foreach by reference (modify original)
foreach ($fruits as &$fruit) {
    $fruit = strtoupper($fruit);
}
unset($fruit); // IMPORTANT: unset reference after loop

// break with levels
for ($i=0; $i<3; $i++) {
    for ($j=0; $j<3; $j++) {
        if ($j==1) break 2; // break both loops
    }
}

// continue with levels
for ($i=0; $i<5; $i++) {
    if ($i==3) continue; // skip 3
    echo $i; // 0 1 2 4
}

```

Chapter 4: Functions

Declarations, types, closures, recursion, generators

Function Declarations

```
<?php
// Basic function
function greet(string $name): string {
    return "Hello, $name!";
}

// Default parameters (must be after required ones)
function power(float $base, int $exp = 2): float {
    return $base ** $exp;
}
power(3); // 9.0
power(3, 3); // 27.0

// Type declarations (PHP 7.0+)
function divide(int|float $a, int|float $b): float {
    if ($b === 0) throw new DivisionByZeroError();
    return $a / $b;
}

// Nullable types
function findUser(?int $id): ?array {
    if ($id === null) return null;
    return ["id" => $id, "name" => "User $id"];
}

// Return type: void, never, mixed
function logMessage(string $msg): void {
    file_put_contents("log.txt", $msg . PHP_EOL, FILE_APPEND);
    // no return statement
}

// Variadic functions (...)
function sumAll(int ...$nums): int {
    return array_sum($nums);
}
sumAll(1, 2, 3, 4, 5); // 15

// Spread operator
$numbers = [1, 2, 3];
sumAll(...$numbers); // 6

// Pass by reference
function addSuffix(string &$str, string $suffix): void {
    $str .= $suffix;
}
$name = "Alice";
addSuffix($name, " Smith");
echo $name; // "Alice Smith"
```

Anonymous Functions & Closures

```
<?php
// Anonymous function
$double = function(int $n): int { return $n * 2; };
echo $double(5); // 10

// Closure – capture outer variables with "use"
$multiplier = 3;
$multiply = function(int $n) use ($multiplier): int {
    return $n * $multiplier;
};
echo $multiply(5); // 15

// Capture by reference
$count = 0;
$increment = function() use (&$count): void { $count++; };
$increment(); $increment();
echo $count; // 2

// Arrow functions (PHP 7.4+) – auto-capture by value
$multiplier = 4;
$fn = fn($n) => $n * $multiplier; // no "use" needed
echo $fn(5); // 20

// Higher-order functions
$nums = [1, 2, 3, 4, 5, 6];
$evens = array_filter($nums, fn($n) => $n % 2 === 0);
$doubled = array_map(fn($n) => $n * 2, $nums);
$sum = array_reduce($nums, fn($carry, $n) => $carry + $n, 0);

// usort with closure
$people = [["name"=>"Bob", "age"=>25], ["name"=>"Alice", "age"=>30]];
usort($people, fn($a, $b) => $a["age"] <=> $b["age"]);
```

Recursion & Generators

```

<?php
// Recursion
function factorial(int $n): int {
    if ($n <= 1) return 1;
    return $n * factorial($n - 1);
}
echo factorial(10); // 3628800

// Memoized recursion
function fib(int $n, array &$memo = []): int {
    if ($n <= 1) return $n;
    if (isset($memo[$n])) return $memo[$n];
    return $memo[$n] = fib($n-1,$memo) + fib($n-2,$memo);
}

// Generator function – memory-efficient sequences
function fibonacci(): Generator {
    [$a, $b] = [0, 1];
    while (true) {
        yield $a;
        [$a, $b] = [$b, $a + $b];
    }
}

$gen = fibonacci();
for ($i = 0; $i < 10; $i++) {
    echo $gen->current() . " ";
    $gen->next();
}
// 0 1 1 2 3 5 8 13 21 34

// Generator with key => value
function indexedRange(int $start, int $end): Generator {
    for ($i = $start; $i <= $end; $i++) {
        yield $i => $i * $i; // key => value
    }
}

foreach (indexedRange(1,5) as $n => $sq) {
    echo "$n^2 = $sq\n";
}

```

Chapter 5: Arrays

All array types, sorting, searching, functional methods

Array Creation & Access

```
<?php
// Indexed array
$fruits = ["apple", "banana", "cherry"];
$fruits = array("apple", "banana", "cherry"); // old syntax

// Associative array
$user = ["name"=>"Alice", "age"=>30, "city"=>"Berlin"];

// Multidimensional
$matrix = [[1,2,3],[4,5,6],[7,8,9]];
$users = [
    ["id"=>1, "name"=>"Alice", "role"=>"admin"],
    ["id"=>2, "name"=>"Bob", "role"=>"user"],
];

// Access
echo $fruits[0]; // apple
echo $user["name"]; // Alice
echo $matrix[1][2]; // 6
echo $users[0]["role"]; // admin

// Destructuring (list/[] syntax)
[$a, $b, $c] = [1, 2, 3];
[, $second] = [10, 20, 30]; // skip first
["name"=>$n, "age"=>$a] = $user;

// Spread in arrays (PHP 8.1)
$merged = [...$fruits, "date", "elderberry"];
$combined = [...$user, "active"=>true];
```

Array Functions — Complete

Array functions - Part 1

```

// Add / Remove
array_push($arr,"x","y"); $arr[]="z"; // add to end
$last=array_pop($arr); $first=array_shift($arr);
array_unshift($arr,"x"); // add to start

// Search
in_array("val",$arr,true); // strict search
array_search("val",$arr); // returns key
array_key_exists("key",$arr); // key exists?

// Slice & Splice
array_slice($arr,1,2); // extract sub-array
array_splice($arr,1,1,["new"]); // replace elements

// Merge & Combine
array_merge($a,$b); // merge (re-index)
array_combine($keys,$vals); // keys + values
$a + $b; // union (keep left)
array_unique($arr); // remove duplicates
array_flip($arr); array_reverse($arr);

```

Array functions - Part 2

```

// Sorting
sort($arr); rsort($arr); // sort values
asort($arr); arsort($arr); // sort, keep keys
ksort($arr); krsort($arr); // sort by key
usort($arr, fn($a,$b)=>$a<=>$b); // custom sort

// Functional
array_map(fn($x)=>$x*2,$arr); // transform
array_filter($arr,fn($x)=>$x>0); // filter
array_reduce($arr,fn($c,$x)=>$c+$x,0); // reduce to value
array_walk($arr,fn(&$v,$k)=>$v=trim($v));

// Math & Stats
array_sum($arr); array_product($arr);
count($arr); array_count_values($arr);

// Set Operations
array_intersect($a,$b); array_diff($a,$b);
array_intersect_key($a,$b); array_diff_key($a,$b);

// Utility
array_fill(0,5,"x"); range(1,10); range("a","e");
array_chunk($arr,3); array_column($users,"name");
array_column($users,"name","id"); // keyed by id

```

Chapter 6: OOP Basics

Classes, objects, visibility, constructors, destructors

Classes & Objects

```
<?php
class BankAccount {
    public string $owner;
    protected string $currency = "EUR";
    private float $balance;
    const INTEREST_RATE = 0.02;
    const MAX_WITHDRAWAL = 5000.0;
    private static int $totalAccounts = 0;

    public function __construct(string $owner, float $balance = 0.0) {
        $this->owner = $owner;
        $this->balance = max(0, $balance);
        self::$totalAccounts++;
    }
    public function __destruct() { self::$totalAccounts--; }

    // Methods returning $this enable method chaining
    public function deposit(float $amount): static {
        if ($amount <= 0) throw new InvalidArgumentException("Positive only");
        $this->balance += $amount;
        return $this;
    }
    public function withdraw(float $amount): static {
        if ($amount > self::MAX_WITHDRAWAL) throw new RangeException("Limit exceeded");
        if ($amount > $this->balance) throw new RuntimeException("Insufficient funds");
        $this->balance -= $amount;
        return $this;
    }
    public function getBalance(): float { return $this->balance; }
    public static function getTotalAccounts(): int { return self::$totalAccounts; }

    public function __toString(): string {
        return sprintf("Account(%s: %.2f %s)", $this->owner, $this->balance, $this->currency);
    }
    public function __clone() { $this->owner = "Clone of ".$this->owner; }
}

$acc = new BankAccount("Alice", 1000.0);
$acc->deposit(500)->withdraw(200); // method chaining
echo $acc; // Account(Alice: 1300.00 EUR)
echo BankAccount::INTEREST_RATE; // 0.02
$clone = clone $acc;
```

Magic Methods

Magic Method	When Called
<code>__construct()</code>	Object is created with new

__destruct()	Object is garbage-collected
__toString()	Object used as string (echo \$obj)
__get(\$name)	Reading inaccessible property
__set(\$name,\$val)	Writing inaccessible property
__isset(\$name)	isset() on inaccessible property
__unset(\$name)	unset() on inaccessible property
__call(\$name,\$args)	Calling inaccessible method
__callStatic(\$n,\$a)	Calling inaccessible static method
__invoke(\$args)	Object used as function: \$obj()
__clone()	Object is cloned with clone keyword
__debugInfo()	var_dump() is called on object
__serialize()	serialize() is called (PHP 7.4+)
__unserialize()	unserialize() is called (PHP 7.4+)

Chapter 7: OOP Advanced

Inheritance, abstract, interfaces, traits, static, readonly

Inheritance

```
<?php
abstract class Shape {
    protected string $color;

    public function __construct(string $color = "black") {
        $this->color = $color;
    }

    // Abstract method – MUST be implemented by children
    abstract public function area(): float;
    abstract public function perimeter(): float;

    // Concrete method – inherited as-is
    public function describe(): string {
        return sprintf("%s %s: area=%.2f, perimeter=%.2f",
            $this->color, static::class,
            $this->area(), $this->perimeter());
    }
}

class Circle extends Shape {
    public function __construct(
        private readonly float $radius,
        string $color = "red"
    ) {
        parent::__construct($color);
    }
    public function area(): float { return M_PI * $this->radius ** 2; }
    public function perimeter(): float { return 2 * M_PI * $this->radius; }
}

class Rectangle extends Shape {
    public function __construct(
        private readonly float $width,
        private readonly float $height,
        string $color = "blue"
    ) { parent::__construct($color); }
    public function area(): float { return $this->width * $this->height; }
    public function perimeter(): float { return 2*($this->width+$this->height); }
}

$shapes = [new Circle(5,"red"), new Rectangle(4,6,"blue")];
foreach ($shapes as $shape) echo $shape->describe() . "\n";
```

Interfaces

```
<?php
// Interfaces define a contract – no implementation
interface Serializable {
    public function serialize(): string;
    public function unserialize(string $data): void;
}

interface Loggable {
    public function log(string $level, string $message): void;
}

// Interface can extend multiple interfaces
interface AdvancedSerializable extends Serializable, Loggable {
    public function toJson(): string;
}

// Class can implement MULTIPLE interfaces
class UserRepository implements Serializable, Loggable {
    public function serialize(): string { return json_encode($this); }
    public function unserialize(string $data): void { /* ... */ }
    public function log(string $level, string $msg): void {
        echo "[$level] $msg\n";
    }
}
```

Traits

```

<?php
// Traits – horizontal code reuse
trait Timestampable {
    private \DateTimeImmutable $createdAt;
    private \DateTimeImmutable $updatedAt;

    public function setCreatedAt(): void {
        $this->createdAt = new \DateTimeImmutable();
    }
    public function touch(): void {
        $this->updatedAt = new \DateTimeImmutable();
    }
    public function getCreatedAt(): string {
        return $this->createdAt->format("Y-m-d H:i:s");
    }
}

trait SoftDeletable {
    private ?\DateTimeImmutable $deletedAt = null;
    public function softDelete(): void {
        $this->deletedAt = new \DateTimeImmutable();
    }
    public function isDeleted(): bool { return $this->deletedAt !== null; }
    public function restore(): void { $this->deletedAt = null; }
}

class Article {
    use Timestampable, SoftDeletable;

    public function __construct(public string $title) {
        $this->setCreatedAt();
    }
}

$a = new Article("PHP 8 Features");
$a->softDelete();
echo $a->isDeleted() ? "Deleted" : "Active";

```

Chapter 8: Namespaces & Composer

Autoloading, PSR-4, packages

Namespaces

```
<?php
// File: src/Models/User.php
namespace App\Models;

use App\Contracts\UserInterface;
use App\Traits\Timestampable;
use \DateTimeImmutable;

class User implements UserInterface {
    use Timestampable;
    public function __construct(
        public readonly int $id,
        public readonly string $name,
        public readonly string $email
    ) { $this->setCreatedAt(); }
}

// File: src/Services/UserService.php
namespace App\Services;

use App\Models\User;
use App\Repositories\UserRepositoryInterface;

class UserService {
    public function __construct(
        private UserRepositoryInterface $repo
    ) {}

    public function findById(int $id): ?User {
        return $this->repo->findById($id);
    }
}

// Aliasing
use App\Models\User as UserModel;
use App\Admin\Models\User as AdminUser;

// Group use declarations (PHP 7+)
use App\Models\{User, Post, Comment};
use function App\Helpers\{formatDate, formatMoney};
use const App\Config\{DEBUG, VERSION};
```

Composer & Autoloading

```
// composer.json
{
  "name": "myapp/myapp",
  "require": {
    "php": "^8.2",
    "guzzlehttp/guzzle": "^7.0",
    "vlucas/phpdotenv": "^5.5",
    "ramsey/uuid": "^4.7"
  },
  "require-dev": {
    "phpunit/phpunit": "^10.0",
    "phpstan/phpstan": "^1.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "autoload-dev": {
    "psr-4": {
      "Tests\\": "tests/"
    }
  }
}

# Terminal commands
composer install # install from composer.lock
composer update # update to latest allowed
composer require vendor/pkg # add new package
composer remove vendor/pkg # remove package
composer dump-autoload # regenerate autoload files
composer show # list installed packages

// In your entry point (public/index.php)
require __DIR__ . '/../vendor/autoload.php';

use App\Models\User;
$user = new User(1, "Alice", "alice@example.com");
```

Chapter 9: Error Handling

Exceptions, custom exceptions, error levels, logging

Exceptions

```

<?php
// Built-in exception hierarchy
// Throwable
// ■■■■ Error (fatal PHP errors)
// ■ ■■■■ TypeError, ValueError, ArithmeticError
// ■ ■■■■ DivisionByZeroError, ParseError
// ■■■■ Exception (application exceptions)
// ■■■■ RuntimeException, LogicException
// ■■■■ InvalidArgumentException, OutOfRangeException
// ■■■■ BadMethodCallException, ...

// Custom exception hierarchy
class AppException extends RuntimeException {}
class ValidationException extends AppException {
    private array $errors;
    public function __construct(array $errors, string $msg="Validation failed") {
        parent::__construct($msg, 422);
        $this->errors = $errors;
    }
    public function getErrors(): array { return $this->errors; }
}
class NotFoundException extends AppException {
    public function __construct(string $resource, int|string $id) {
        parent::__construct("$resource #{$id} not found", 404);
    }
}

// Try / catch / finally
function processUser(int $id): array {
    try {
        if ($id <= 0) throw new InvalidArgumentException("ID must be positive");
        $user = findUser($id) ?? throw new NotFoundException("User", $id);
        return $user;
    } catch (ValidationException $e) {
        // Handle validation errors
        return ["errors" => $e->getErrors()];
    } catch (NotFoundException $e) {
        http_response_code(404);
        return ["error" => $e->getMessage()];
    } catch (AppException $e) {
        error_log("AppException: " . $e->getMessage());
        return ["error" => "Application error"];
    } catch (\Throwable $e) {
        error_log("Unexpected: " . $e->getMessage());
        return ["error" => "Internal server error"];
    } finally {
        // Always runs: cleanup, closing connections, logging
        closeDatabaseConnection();
    }
}

// Throw as expression (PHP 8)
$value = $input ?? throw new InvalidArgumentException("Required");
$arr = $data ?: throw new RuntimeException("Empty data");

```


Chapter 10: File System

Reading, writing, directories, uploads, streams

File Reading & Writing

```
<?php
// Simple read/write
$content = file_get_contents("data.txt");
file_put_contents("out.txt", "Hello!\n");
file_put_contents("log.txt", "Entry\n", FILE_APPEND | LOCK_EX);

// Line by line (memory-efficient for large files)
$lines = file("data.txt", FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);

// Low-level file handling
$handle = fopen("data.csv", "r");
while (!feof($handle)) {
    $line = fgets($handle, 4096); // read up to 4KB
    $row = fgetcsv($handle); // parse as CSV
    $data = fread($handle, 1024); // read N bytes
}
fclose($handle);

// Write modes: w=overwrite, a=append, x=exclusive, r+=read+write
$handle = fopen("out.txt", "w");
fwrite($handle, "Line 1\n");
fputcsv($handle, ["Alice", 30, "Berlin"]);
fclose($handle);

// File info
file_exists("file.txt"); // true/false
is_file("file.txt"); // is regular file
is_dir("folder"); // is directory
is_readable("file.txt"); // readable
is_writable("file.txt"); // writable
filesize("file.txt"); // size in bytes
filemtime("file.txt"); // last modified (timestamp)
pathinfo("path/file.txt"); // ["dirname", "basename", "extension", "filename"]
basename("path/file.txt"); // "file.txt"
dirname("path/file.txt"); // "path"
realpath("../file.txt"); // absolute path

// File operations
copy("src.txt", "dst.txt");
rename("old.txt", "new.txt"); // move or rename
unlink("file.txt"); // delete file
chmod("file.txt", 0644); // change permissions

// Temporary files
$tmpFile = tempnam(sys_get_temp_dir(), "php_");
$tmpHandle = tmpfile(); // auto-deleted on close
```

Directory Operations

```
<?php
mkdir("new_folder", 0755, true); // recursive
rmdir("empty_folder"); // must be empty
chdir("/var/www"); // change directory
getcwd(); // current directory

// List directory contents
$files = scandir("."); // array of filenames
$files = array_diff(scandir("."), [".", ".."]); // skip . and ..

// Recursive directory listing with glob()
$phpFiles = glob("src/**/*.php", GLOB_BRACE);
$images = glob("uploads/*.{jpg,png,gif}", GLOB_BRACE);

// Using RecursiveDirectoryIterator
$iter = new RecursiveIteratorIterator(
    new RecursiveDirectoryIterator("src/")
);
foreach ($iter as $file) {
    if ($file->isFile() && $file->getExtension() === "php") {
        echo $file->getPathname() . "\n";
    }
}
```

Chapter 11: Superglobals & Forms

`$_GET`, `$_POST`, `$_FILES`, validation, sanitization

Superglobal Variables

Variable	Contents	Common Use
<code>\$_GET</code>	URL query parameters	Search forms, pagination
<code>\$_POST</code>	HTTP POST body	Login, registration forms
<code>\$_REQUEST</code>	<code>\$_GET</code> + <code>\$_POST</code> + <code>\$_COOKIE</code>	General form data
<code>\$_FILES</code>	Uploaded files info	File upload forms
<code>\$_SERVER</code>	Server & request info	Request method, IP, headers
<code>\$_SESSION</code>	Session data (server-side)	Login state, cart
<code>\$_COOKIE</code>	HTTP cookies	Remember me, preferences
<code>\$_ENV</code>	Environment variables	Config, secrets
<code>\$GLOBALS</code>	All global variables	Global scope access

Complete Form Handling

```

<?php
// Validation helper functions
function sanitizeString(string $input): string {
    return htmlspecialchars(trim(strip_tags($input)), ENT_QUOTES, "UTF-8");
}

function validateEmail(string $email): bool {
    return (bool)filter_var($email, FILTER_VALIDATE_EMAIL);
}

function validateUrl(string $url): bool {
    return (bool)filter_var($url, FILTER_VALIDATE_URL);
}

// Process form
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $errors = [];

    // Get and sanitize
    $name = sanitizeString($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $age = filter_input(INPUT_POST, "age", FILTER_VALIDATE_INT,
        ["options"=>["min_range"=>1,"max_range"=>120]]);

    // Validate
    if (strlen($name) < 2) $errors[] = "Name too short";
    if (!validateEmail($email)) $errors[] = "Invalid email";
    if ($age === false) $errors[] = "Invalid age (1-120)";

    if (empty($errors)) {
        // Process valid data
        echo "Success: $name, $email, $age";
    } else {
        foreach ($errors as $error) echo "<p class=error>$error</p>";
    }
}

// filter_var examples
filter_var("42", FILTER_VALIDATE_INT); // 42 or false
filter_var("3.14", FILTER_VALIDATE_FLOAT); // 3.14 or false
filter_var(" hi ", FILTER_SANITIZE_STRING); // "hi"
filter_var("<b>Hi</b>", FILTER_SANITIZE_SPECIAL_CHARS);

```

Chapter 12: Sessions & Cookies

State management between requests

Sessions

```
<?php
// MUST be first thing before any output
session_start();

// Store data
$_SESSION["user_id"] = 42;
$_SESSION["username"] = "Alice";
$_SESSION["role"] = "admin";
$_SESSION["logged_in"] = true;
$_SESSION["cart"] = [];

// Read data
if (isset($_SESSION["logged_in"]) && $_SESSION["logged_in"]) {
    echo "Welcome, " . $_SESSION["username"];
}

// Add to session array
$_SESSION["cart"][] = ["id"=>1, "qty"=>2];

// Session info
session_id(); // get current session ID
session_name(); // get/set session name (default: PHPSESSID)

// Regenerate ID (prevent session fixation)
session_regenerate_id(true); // true = delete old session

// Destroy session (logout)
$_SESSION = []; // clear array
if (ini_get("session.use_cookies")) { // delete cookie
    $params = session_get_cookie_params();
    setcookie(session_name(), "", time()-3600,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]);
}
session_destroy();

// Session configuration (php.ini or ini_set)
ini_set("session.gc_maxlifetime", 3600); // 1 hour
ini_set("session.cookie_secure", 1); // HTTPS only
ini_set("session.cookie_httponly", 1); // no JS access
ini_set("session.cookie_samesite", "Lax");
```

Cookies

```
<?php
// Set cookie – MUST be before any output
setcookie(
    "username", // name
    "Alice", // value
    [
        "expires" => time() + 86400 * 30, // 30 days
        "path" => "/",
        "domain" => "example.com",
        "secure" => true, // HTTPS only
        "httponly" => true, // no JavaScript access
        "samesite" => "Lax", // CSRF protection
    ]
);

// Read cookie
$username = $_COOKIE["username"] ?? "Guest";

// Delete cookie (set expiry in past)
setcookie("username", "", [
    "expires" => time() - 3600,
    "path" => "/",
]);

// JSON in cookie
setcookie("prefs", json_encode(["theme"=>"dark", "lang"=>"en"]),
    time() + 86400 * 365);
$prefs = json_decode($_COOKIE["prefs"] ?? "{}", true);
```

Chapter 13: PDO & MySQL

Connections, prepared statements, CRUD, transactions

PDO Connection & Configuration

```
<?php
// Best practice: singleton connection
class Database {
    private static ?PDO $instance = null;

    public static function getInstance(): PDO {
        if (self::$instance === null) {
            $dsn = sprintf(
                "mysql:host=%s;port=%s;dbname=%s;charset=utf8mb4",
                $_ENV["DB_HOST"] ?? "localhost",
                $_ENV["DB_PORT"] ?? "3306",
                $_ENV["DB_NAME"] ?? "mydb"
            );
            self::$instance = new PDO($dsn,
                $_ENV["DB_USER"] ?? "root",
                $_ENV["DB_PASS"] ?? "",
                [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                    PDO::ATTR_EMULATE_PREPARES => false,
                    PDO::ATTR_PERSISTENT => true,
                ]
            );
        }
        return self::$instance;
    }
}

$db = Database::getInstance();
```

CRUD with Prepared Statements

```

<?php
$db = Database::getInstance();

// CREATE
$stmt = $db->prepare(
    "INSERT INTO users (name, email, age, created_at)",
    "VALUES (:name, :email, :age, NOW())"
);
$stmt->execute([":name"=>"Alice", ":email"=>"alice@x.com", ":age"=>30]);
$newId = (int)$db->lastInsertId();

// READ – fetch one
$stmt = $db->prepare("SELECT * FROM users WHERE id = :id");
$stmt->execute([":id" => $newId]);
$user = $stmt->fetch(); // FETCH_ASSOC (set in constructor)

// READ – fetch all
$stmt = $db->prepare(
    "SELECT * FROM users WHERE city = :city ORDER BY name LIMIT :limit"
);
$stmt->bindValue(":city", "Berlin", PDO::PARAM_STR);
$stmt->bindValue(":limit", 10, PDO::PARAM_INT);
$stmt->execute();
$users = $stmt->fetchAll();

// Fetch as object
$user = $stmt->fetchObject(User::class);

// UPDATE
$stmt = $db->prepare(
    "UPDATE users SET name=:name, age=:age WHERE id=:id"
);
$affected = $stmt->execute([":name"=>"Bob", ":age"=>25, ":id"=>1]);
echo $stmt->rowCount() . " rows updated";

// DELETE
$stmt = $db->prepare("DELETE FROM users WHERE id = :id");
$stmt->execute([":id" => 5]);

// Transactions
try {
    $db->beginTransaction();
    $db->prepare("UPDATE accounts SET balance=balance-:amt WHERE id=:id")
    ->execute([":amt"=>500, ":id"=>1]);
    $db->prepare("UPDATE accounts SET balance=balance+:amt WHERE id=:id")
    ->execute([":amt"=>500, ":id"=>2]);
    $db->commit();
    echo "Transfer successful";
} catch (\Throwable $e) {
    $db->rollBack();
    throw new RuntimeException("Transfer failed: " . $e->getMessage());
}

```


Chapter 14: Security & Regex

XSS, SQL injection, CSRF, password hashing, headers

Password Hashing

```
<?php
// NEVER store plain passwords!
// NEVER use md5() or sha1() for passwords!

// Hash password (bcrypt by default)
$hash = password_hash($password, PASSWORD_BCRYPT, ["cost"=>12]);
$hash = password_hash($password, PASSWORD_ARGON2ID); // PHP 7.3+

// Verify password
if (password_verify($inputPassword, $storedHash)) {
    echo "Login successful";
}

// Rehash if algorithm/cost changes
if (password_needs_rehash($hash, PASSWORD_ARGON2ID)) {
    $hash = password_hash($password, PASSWORD_ARGON2ID);
    // Update hash in database
}
```

XSS Prevention

```
<?php
// ALWAYS escape output before displaying user data
function esc(string $str): string {
    return htmlspecialchars($str, ENT_QUOTES | ENT_HTML5, "UTF-8");
}

// Use in templates
echo "<p>" . esc($userInput) . "</p>";

// Content Security Policy header
header("Content-Security-Policy: default-src 'self'; script-src 'self'");
header("X-XSS-Protection: 1; mode=block");
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: DENY");
```

CSRF Protection

```
<?php
session_start();

// Generate CSRF token
function generateCsrfToken(): string {
    if (empty($_SESSION["csrf_token"])) {
        $_SESSION["csrf_token"] = bin2hex(random_bytes(32));
    }
    return $_SESSION["csrf_token"];
}

// Validate CSRF token
function validateCsrfToken(string $token): bool {
    return isset($_SESSION["csrf_token"])
        && hash_equals($_SESSION["csrf_token"], $token);
}

// In form
$token = generateCsrfToken();
// <input type="hidden" name="csrf_token" value="<?= esc($token) ?>">

// In handler
if (!validateCsrfToken($_POST["csrf_token"] ?? "")) {
    http_response_code(403);
    die("CSRF validation failed");
}
```

Chapter 15: Exercises

20 professional exercises covering all topics

Exercises

FILL

Ex 1. What PHP function sanitizes output to prevent XSS?

Hint: Think output encoding

■ **Answer:** `htmlspecialchars(str, ENT_QUOTES, UTF-8)`

FILL

Ex 2. Name three PHP array functions that accept a callback.

■ **Answer:** `array_map()`, `array_filter()`, `array_walk()`, `usort()`

TRUEFALSE

Ex 3. PHP match uses loose comparison like switch. (True/False)

■ **Answer:** FALSE – match uses strict comparison (`===`). switch uses loose (`==`).

FILL

Ex 4. What is the difference between `self::` and `static::` in PHP OOP?

■ **Answer:** `self::` refers to the class where code is written (early binding). `static::` uses late static binding – refers to the class actually called at runtime.

WRITE

Ex 5. Write a PHP function that validates a strong password (min 8 chars, uppercase, digit, special char).

■ **Answer:**

```
function isStrongPassword(string p): bool { return strlen(p)>=8 && preg_match('/[A-Z]/',p) && preg_match('/[0-9]/',p) && preg_match('/^[a-zA-Z0-9]/',p); }
```

WRITE

Ex 6. Write a PHP class Stack with `push()`, `pop()`, `peek()`, `isEmpty()`, `size()`.

■ **Answer:**

```
class Stack { private array data=[]; push(v):void{data[]=v;} pop():mixed{return array_pop(data);} peek():mixed{return end(data);} isEmpty():bool{return empty(data);} size():int{return count(data);} }
```

FILL

Ex 7. What PDO attribute throws exceptions on database errors?

■ **Answer:** `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION`

DEBUG

Ex 8. Bug: prepare('SELECT * FROM users WHERE id = variable') execute() — what is wrong?

■ **Answer:** Variable interpolation in SQL! Use named placeholder ':id' and execute([':id'=>id]) to prevent SQL injection.

WRITE

Ex 9. Write a PHP trait HasUuid that auto-generates a UUID v4.

■ **Answer:**

```
trait HasUuid { public readonly string uuid; public function initUuid():void {  
    uuid = sprintf('%04x%04x-%04x-%04x-%04x-%012x', mt_rand(0,0xffff), mt_rand(0,0xffff),  
    mt_rand(0,0xffff), mt_rand(0,0xffff)|0x4000, mt_rand(0,0x3fff)|0x8000,  
    mt_rand(0,0xffffffffffffffff)); } }
```

EXPLAIN

Ex 10. Explain require, include, require_once, include_once differences.

■ **Answer:** require: fatal error if missing. include: warning if missing (continues). _once variants: skip if already included. Use require_once for class/function files to avoid redeclaration errors.

WRITE

Ex 11. Write a PHP function that recursively flattens a nested array.

■ **Answer:**

```
function flatten(array arr):array { result=[];  
    array_walk_recursive(arr,function(v)use(&result){result[]=$v;}); return result; }
```

FILL

Ex 12. What function securely hashes passwords in PHP?

■ **Answer:** password_hash() with PASSWORD_BCRYPT or PASSWORD_ARGON2ID

WRITE

Ex 13. Write a PHP generator yielding prime numbers up to a limit.

■ **Answer:**

```
function primes(int limit): Generator { for(n=2;n<=limit;n++){ isPrime=true;  
    for(i=2;i<=sqrt(n);i++){ if(n%i===0){isPrime=false;break;} } if(isPrime) yield n; } }
```

EXPLAIN

Ex 14. Difference between abstract classes and interfaces in PHP?

■ **Answer:** Abstract classes: can have implementation, properties, constructor. One parent only. Interfaces: only signatures (no implementation). Multiple interfaces allowed. Use abstract for is-a; interfaces for contracts/capabilities.

WRITE

Ex 15. Write a PSR-4 autoloader using spl_autoload_register.

■ **Answer:** `spl_autoload_register(function(class):void { prefix='App\'; baseDir=__DIR__.'/src/'; if(strncmp(prefix,class,len)!=0)return; file=baseDir.str_replace('\','/',substr(class,len)).'.php'; if(file_exists(file)) require file; });`

DEBUG

Ex 16. Is `unset(item)` necessary after: `foreach(items as item) { results[]=process(item); }?`

■ **Answer:** NO – `unset` is only needed after `foreach` by REFERENCE (`foreach(arr as &val)`). Without the `&`, no reference is created.

WRITE

Ex 17. Write a PHP function that reads a CSV file and returns an array of associative arrays.

■ **Answer:** `function csvToArray(string file):array { rows=[]; if((h=fopen(file,'r'))!=false){ headers=fgetcsv(h); while((row=fgetcsv(h))!=false) rows[]=array_combine(headers,row); fclose(h); } return rows; }`

FILL

Ex 18. What is the purpose of `session_regenerate_id(true)?`

■ **Answer:** Creates a new session ID and deletes the old session file, preventing session fixation attacks.

WRITE

Ex 19. Write a `UserRepository` class using PDO with `findById` and `save` methods.

■ **Answer:** `class UserRepository { __construct(private PDO db){} findById(int id):?array { s=db->prepare('SELECT*FROM users WHERE id=:id'); s->execute([':id'=>id]); return s->fetch()?:null; } save(array user):int { s=db->prepare('INSERT INTO users(name,email)VALUES(:n,:e)'); s->execute([':n'=>user[name],':e'=>user[email]]); return(int)db->lastInsertId(); } }`

EXPLAIN

Ex 20. Explain PHP 8 named arguments and their benefits.

■ **Answer:** Named arguments: `array_slice(array:arr, offset:1, length:3)`. Skip optional params, self-documenting, order-independent. Works with built-in and user-defined functions.